Manual Técnico: Parque Residencial, Sistema de Gestión de Parqueo

Introducción a la Programación y Computación 1, Segundo Semestre 2026 Universidad de San Carlos de Guatemala, Facultad de Ingeniería

Autor: Jonathan Enmanuel Gutierrez Estrada 202401690 **Fecha:** 30 de octubre de 2026
Lenguaje: Java JDK 17, interfaz con Java Swing

1. Descripción general

El sistema simula el flujo de vehículos de un residencial: llegan a una cola de entrada, son atendidos por una de tres garitas, reciben un espacio según su tipo de usuario, permanecen estacionados, piden salir, hacen fila en la cola de salida y son atendidos por una de dos garitas que liberan el espacio. Cada acción queda registrada en una pila de eventos.

Todas las estructuras de datos (listas, colas y pila) fueron programadas desde cero. No se usan `ArrayList`, `LinkedList`, `Stack`, `Queue`, `Deque` ni otras colecciones de `java.util` para reemplazarlas. La interfaz usa solo Java Swing, escrita a mano, sin JavaFX, bases de datos ni frameworks externos.

2. Arquitectura y organización de paquetes

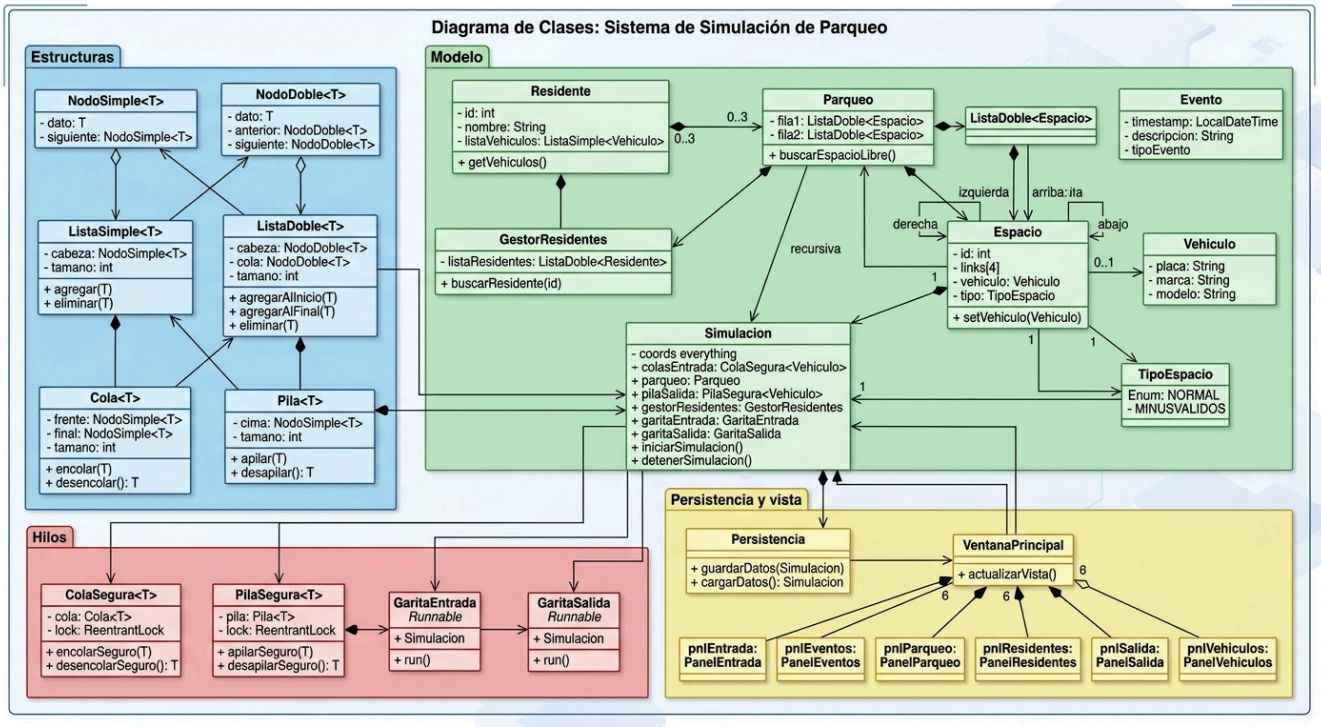
El código se divide en cinco paquetes con responsabilidades separadas:

Paquete	Responsabilidad	Clases
`estructuras`	Estructuras genéricas, sin conocimiento del negocio	`NodoSimple<T>`, `NodoDoble<T>`, `ListaSimple<T>`, `ListaDoble<T>`, `Cola<T>`, `Pila<T>`
`modelo`	Datos y reglas del negocio	`Residente`, `Vehiculo`, `Espacio`, `TipoEspacio`, `Parqueo`, `GestorResidentes`, `Evento`, `Simulacion`
`hilos`	Concurrencia y sincronización	`ColaSegura<T>`, `PilaSegura<T>`, `GaritaEntrada`, `GaritaSalida`
`persistencia`	Lectura y escritura de archivos de texto	`Persistencia`

Paquete	Responsabilidad	Clases
`vista`	Interfaz gráfica Swing	`VentanaPrincipal`, `PanelParqueo`, `PanelResidentes`, `PanelVehiculos`, `PanelEntrada`, `PanelSalida`, `PanelEventos`

Decisión clave: las estructuras son **genéricas** (`<T>`). Una `Cola<T>` no sabe si guarda vehículos o textos, lo que permite reutilizarla para la cola de entrada y la de salida, y hace que el compilador rechace tipos incorrectos en lugar de fallar en ejecución con un `ClassCastException`.

2.1 Diagrama de clases



Relaciones principales que debe mostrar el diagrama:

- `Residente` contiene una `ListaSimple<Vehiculo>` (de 0 a 3 elementos).
- `GestorResidentes` contiene una `ListaDoble<Residente>`.
- `Parqueo` contiene dos `ListaDoble<Espacio>` (área de socios y área general); cada elemento es el primer espacio de una fila.
- `Espacio` tiene cuatro referencias a otros espacios (izquierda, derecha, arriba, abajo) y una referencia a un `Vehiculo`.
- `ColaSegura<T>` envuelve una `Cola<T>`; `PilaSegura<T>` envuelve una `Pila<T>`.
- `GaritaEntrada` y `GaritaSalida` implementan `Runnable`.

- `Simulacion` coordina las dos colas seguras, el parqueo, la pila segura de eventos, el gestor y las cinco garitas.
- `VentanaPrincipal` contiene los seis paneles.

3. Estructuras de datos

3.1 Nodos

`NodoSimple<T>` guarda un dato y una referencia `siguiente`. `NodoDoble<T>` agrega la referencia `anterior`. El último nodo apunta a `null`.

3.2 Lista simplemente enlazada (`ListaSimple<T>`)

Usada para los vehículos de cada residente. Mantiene `cabeza` y `tamano`. Operaciones: `insertarAlInicio`, `insertarAlFinal`, `buscar`, `eliminar`, `recorrer`, `obtenerEn` y `getTamano`. En `eliminar` se distinguen tres casos: lista vacía, dato en la cabeza y dato en el medio o al final (con un puntero `anterior` que va un paso detrás de `actual`).

3.3 Lista doblemente enlazada (`ListaDoble<T>`)

Usada para los residentes y como índice de filas del parqueo. Mantiene `cabeza`, `cola` y `tamano`. Operaciones: `insertarAlInicio`, `insertarAlFinal`, `buscar`, `eliminar`, `recorrer`, `recorrerAlReves`, `obtenerEn`, `obtenerPrimero` y `getTamano`. En `eliminar`, dos bloques `if/else` independientes actualizan el vecino izquierdo (o `cabeza`) y el vecino derecho (o `cola`), lo que cubre cabeza, cola, medio y lista de un solo elemento sin código repetido.

Regla aplicada en todas las inserciones: **enlazar primero y mover el puntero de control después** (`cola = nuevo` va al final). Si se mueve antes, se pierde la referencia al último nodo real.

3.4 Cola FIFO (`Cola<T>`)

Usa `frente` y `fin`. `encolar` inserta por el final y `desencolar` saca por el frente. Cuando `desencolar` deja la cola vacía, `fin` debe ponerse en `null`; de lo contrario `fin` queda apuntando a un nodo que ya no pertenece a la cola y el siguiente `encolar` engancha el nuevo nodo a ese nodo muerto. `desencolar` devuelve `null` si la cola está vacía. Se agregó `obtenerEn` para que la interfaz pueda mostrar la cola sin sacar elementos.

3.5 Pila LIFO (`Pila<T>`)

Usa solo `tope` . `apilar` inserta al inicio y `desapilar` quita el tope. `verTope` consulta sin modificar. Se agregó `obtenerDesdeTope(posicion)` , donde 0 es el evento más reciente, para mostrar el historial sin vaciarlo.

4. El parqueo: lista doble de listas simples

4.1 Distribución física

Área	Filas	Espacios por fila	Total	Tipo
Socios	A, B, C	25 (A1 a A25, B1 a B25, C1 a C25)	75	`SOCIO`
General y visitantes	E, F, G, H, I	15 (E1 a E15, ..., I1 a I15)	75	`GENERAL`

Total: 150 espacios. No existe la fila D.

4.2 Interpretación adoptada

El enunciado pide una "lista doblemente enlazada de listas simples en la que cada nodo es un espacio físico". La interpretación implementada es:

- Cada **fila** es una cadena de espacios enlazados horizontalmente con `izquierda` y `derecha` .
- Las **filas** se enlazan verticalmente entre sí, espacio por espacio, con `arriba` y `abajo` .
- Un `ListaDoble<Espacio>` por área guarda el primer espacio de cada fila, y funciona como índice de filas.

Cada espacio es un nodo con cuatro enlaces. El área de socios y el área general son estructuras independientes: la fila C no se enlaza con la fila E.

4.3 Construcción

`construirFila` crea los espacios de una fila enlazándolos con `derecha` e `izquierda` y devuelve el primero. `enlazarFilas` recorre dos filas vecinas **a la vez** con dos punteros que avanzan juntos, y une cada espacio con el que tiene encima o debajo (`a.abajo = b` , `b.arriba = a`). Si los punteros no avanzaran juntos, varios nodos superiores apuntarían al mismo nodo inferior y se perdería la alineación por columnas.

Verificación: `contarEspacios()` recorre ambas áreas usando solo los enlaces (`derecha` y `abajo`) y debe devolver 150; esto valida a la vez las filas y los enlaces verticales.

4.4 Operaciones del parqueo

- `buscarLibre(tipo)`: recorre el área indicada y devuelve el primer espacio libre, o `null` si está llena. Quien lo llame debe revisar el `null`.
- `asignar(vehiculo, tipo)`: busca un libre y lo ocupa.
- `asignarSegunTipo(vehiculo, esSocio)`: aplica la regla de asignación (sección 5).
- `liberar(placa)`: busca el espacio que contiene esa placa (primero en socios, después en general, ya que una placa solo puede estar en un espacio) y lo libera. Devuelve el espacio, o `null` si la placa no estaba estacionada.
- `estaEstacionada(placa)`: evita que la misma placa ocupe dos espacios.

Todos los métodos que modifican el estado son `synchronized` (sección 7).

5. Modelo y reglas de negocio

5.1 Entidades

- **Vehiculo:** placa, marca, modelo, color, tipo y estado (`FUERA`, `EN_COLA`, `ESTACIONADO`). `equals` compara por placa, de modo que `buscar` en una lista detecta placas repetidas. Sin ese `equals`, Java compara direcciones de memoria y dos objetos con la misma placa se considerarían distintos.
- **Residente:** id, nombre, casa/lote, indicador de socio y `ListaSimple<Vehiculo>`. `equals` compara por id. El id no tiene setter porque es la identidad del residente.
- **Espacio:** identificador (por ejemplo A12), tipo, estado libre/ocupado, vehículo estacionado y los cuatro enlaces. `ocupar` y `liberar` cambian estado y vehículo juntos, para que nunca quede un espacio ocupado sin vehículo.
- **Evento:** descripción y hora, para la pila de historial.
- **Visitante:** no existe como clase. Es un `Vehiculo` cuya placa no pertenece a ningún residente del gestor.

5.2 Máximo de tres vehículos

`Residente.agregarVehiculo` rechaza el vehículo si `getTamano()` ya alcanzó la constante `MAX_VEHICULOS = 3` o si la placa ya está registrada. Se usa la constante, no un número suelto.

5.3 Regla de asignación de espacios (decisión de diseño)

> Un residente **socio** intenta primero el área A-C. Si está llena, usa el área general E-I. Un residente **no socio** o un **visitante** solo puede usar el área general. Ningún vehículo no socio puede ocupar un espacio A-C.

La protección es estructural: `GaritaEntrada` calcula `esSocio` a partir del dueño del vehículo (`buscarDuenoPorPlaca`), y solo si es `true` se llama a `asignar` con `TipoEspacio.SOCIO` . Un no socio nunca llega al área exclusiva. Si `esSocio` estuviera fijo en `true` , cualquier vehículo caería primero en A-C y se violaría la regla del enunciado (error que se detectó y corrigió durante las pruebas).

5.4 Gestión de residentes

`GestorResidentes` encapsula una `ListaDoble<Residente>` y ofrece `registrar` (rechaza ID repetido), `buscarPorId` , `buscarDuenoPorPlaca` , `modificar` (nombre, casa y socio), `eliminar` , `obtenerEn` y `getTotal` .

5.5 Visitantes

Los visitantes son información temporal. Se registran al llegar (se apila el evento "Visitante registrado"), reciben un espacio del área general y salen como cualquier vehículo. Nunca se agregan al `GestorResidentes` , por lo que no se persisten: la exclusión es estructural, porque `Persistencia.guardar` solo recorre los residentes y sus vehículos.

6. Flujo de la simulación

6.1 Entrada

1. El vehículo llega (`Simulacion.llegaVehiculo`) y se encola en la cola de entrada.
2. Una de las tres garitas lo desencola. Si no hay vehículos, la garita espera con `wait()` .
3. La garita marca el vehículo como en atención y pausa 800 ms para que la simulación sea observable.
4. Se busca al dueño por placa y se calcula `esSocio` .
5. Se llama a `parqueo.asignarSegunTipo` .
6. Si hay espacio, el vehículo pasa a `ESTACIONADO` y se apila "asignado al espacio X". Si no, se apila "rechazado (parqueo lleno)".

6.2 Salida

1. El usuario pide salida (`Simulacion.pideSalida`) y el vehículo se encola en la cola de salida.
2. Una de las dos garitas de salida lo desencola, pausa 800 ms y llama a `parqueo.liberar(placa)`.
3. Si el espacio existía, el vehículo pasa a `FUERA` y se apila "salió, espacio X liberado". Si no, se apila "no estaba estacionado".

7. Hilos y sincronización

7.1 Hilos

Cada garita es un `Runnable` ejecutado en su propio `Thread`: tres de entrada y dos de salida, cinco en total. Todos tienen una función real: consumen de una cola compartida y modifican el parqueo.

7.2 `ColaSegura<T>`

Envuelve una `Cola<T>` y protege sus operaciones con `synchronized`. `desencolar` espera mientras la cola está vacía:

```
public synchronized T desencolar() throws InterruptedException {
    while (cola.estaVacia()) {
        wait();
    }
    return cola.desencolar();
}
```

`encolar` llama a `notifyAll()` para despertar a las garitas dormidas.

Por qué `while` y no `if`: si despiertan tres garitas por un solo vehículo, la primera lo toma y las otras dos encuentran la cola vacía. Con `while` vuelven a evaluar la condición y se duermen otra vez. Con `if` intentarían sacar de una cola vacía y obtendrían `null`.

`wait()` libera el candado del `synchronized` mientras duerme, lo que permite que otro hilo entre a `encolar`.

7.3 Condición de carrera en el parqueo

`asignar` realiza dos pasos: buscar un espacio libre y ocuparlo. Sin sincronización, dos garitas podrían ver el mismo espacio libre antes de que alguna lo ocupe, y la segunda sobrescribiría a la primera. Por eso `asignar`, `asignarSegunTipo`, `liberar` y

`estaEstacionada` son `synchronized`. `asignarSegunTipo` es un solo bloque protegido: el intento en socios y el intento en general ocurren sin que otra garita se interponga.

Para demostrar el fenómeno se incluyó `PruebaCarrera`: dos hilos incrementan un contador 100 000 veces cada uno. Sin `synchronized` el resultado es menor a 200 000 porque `contador++` esconde tres pasos (leer, sumar, escribir). Con `synchronized` siempre da 200 000.

7.4 `PilaSegura<T>`

Envuelve una `Pila<T>` con métodos `synchronized`, porque las garitas apilan eventos mientras la interfaz los lee. No usa `wait()` porque apilar y consultar nunca bloquean.

7.5 `volatile`

Cada garita expone `enAtencion` (el vehículo que atiende, o `null` si está libre). Lo escribe el hilo de la garita y lo lee el hilo de Swing. Se declara `volatile` para garantizar que el lector vea el último valor escrito y no una copia en caché. No hace falta `synchronized` porque es una asignación simple de referencia, no una operación compuesta.

7.6 Detención

`Simulacion.detener()` llama a `interrupt()` en cada hilo. Eso despierta a los hilos dormidos en `wait()` o `sleep()` y los hace salir por el `catch (InterruptedException)`.

8. Interfaz gráfica

La ventana principal (`VentanaPrincipal`) usa un `JTabbedPane` con seis pestañas:

Pestaña	Contenido
Parqueo	Cuadrícula de 150 espacios dentro de un `JScrollPane`: verde = libre, rojo = ocupado, mostrando la placa
Residentes	Registrar, buscar, modificar y eliminar residentes
Vehículos	Asociar vehículos a un residente, mostrarlos, hacer llegar un vehículo (residente o visitante) y pedir salida
Entrada	Contenido de la cola de entrada y estado de las tres garitas

Pestaña	Contenido
Salida	Contenido de la cola de salida y estado de las dos garitas
Eventos	Últimos 50 eventos de la pila, el más reciente arriba

La vista se actualiza con un `javax.swing.Timer` cada 500 ms, que lee las estructuras y repinta. Todo el código Swing se ejecuta en el hilo de eventos (`SwingUtilities.invokeLater`), y los hilos de las garitas nunca tocan componentes gráficos.

Decisión: la vista lee el estado del parqueo sin bloquearlo. Un cuadro puede mostrarse con hasta 500 ms de retraso, lo cual es aceptable porque solo lee y repinta.

Validaciones en la interfaz: ID y nombre obligatorios al registrar, placa obligatoria, aviso al duplicar un ID, aviso al exceder tres vehículos o repetir una placa, y aviso si la placa ya está estacionada.

9. Persistencia

9.1 Formato de los archivos

Se usan dos archivos de texto plano, con `|` como separador. Los archivos se leen y escriben en la carpeta desde la que se ejecuta el programa.

```
residentes.txt:  ID|NOMBRE|CASA|SOCIO
vehiculos.txt:   PLACA|MARCA|MODELO|COLOR|ID_RESIDENTE
```

Ejemplo:

```
R1|Ana|Casa 5|true
R2|Luis|Casa 8|false
```

9.2 Guardado

Se guarda al cerrar la ventana, mediante un `WindowListener` (`windowClosing`), con `DO_NOTHING_ON_CLOSE` para controlar el cierre. El orden es: guardar, detener las garitas y salir. Guardar va primero porque `System.exit(0)` termina el programa de inmediato y lo que esté después no se ejecuta. `guardar` recorre el gestor y escribe una línea por residente y una por vehículo, incluyendo el `ID_RESIDENTE` del dueño.

9.3 Carga

Al iniciar, `cargar` reconstruye las listas en dos fases, y el orden importa:

1. Se lee `residentes.txt` y se registra cada residente en el gestor.
2. Se lee `vehiculos.txt`; para cada línea se busca al dueño con `buscarPorId(ID_RESIDENTE)` y se llama a `agregarVehiculo`.

Si se leyeran primero los vehículos, los dueños aún no existirían y los vehículos se perderían en silencio, porque `if (dueno != null)` los descartaría. Si los archivos no existen (primera ejecución), se devuelve un gestor vacío. Las líneas en blanco se ignoran.

9.4 Decisiones sobre persistencia

- El formato sugerido no incluye el tipo de vehículo, por lo que al cargar se asigna `"carro"` por defecto.
- Los visitantes y los vehículos estacionados **no** se persisten. Al reiniciar, el parqueo empieza vacío.

10. Decisiones de diseño y limitaciones

Tema	Decisión
Regla de socios	Socio: A-C y, si está lleno, general. No socio y visitante: solo general
Lista de listas	Filas como nodos de una `ListaDoble`; espacios con cuatro enlaces
Tipo de vehículo al cargar	`"carro"` por defecto
Lectura de la vista	Sin bloqueo, con hasta 500 ms de retraso
Validación de datos	En los paneles; el modelo asume datos válidos
Eliminar un residente con vehículo estacionado	El espacio permanece ocupado hasta que el vehículo solicite salida
Placa repetida	Se rechaza si ya está estacionada

Limitaciones conocidas: `buscarPorId` llama a `obtenerEn(i)` dentro de un ciclo, lo que recorre la lista varias veces (aceptable para pocos residentes; se priorizó la claridad sobre la optimización). La salida de un vehículo se identifica por placa, por lo que los demás datos del formulario se ignoran al pedir salida.

11. Pruebas realizadas

#	Prueba	Resultado esperado y obtenido
1	Estructuras por consola	Salidas correctas en las cuatro estructuras
2	`contarEspacios()`	150
3	Enlaces verticales ida y vuelta	`A1.abajo.abajo.arriba` es B1
4	Tres garitas, 10 vehículos	10 eventos, sin espacios repetidos
5	Cinco garitas, 6 entradas y 6 salidas	12 eventos, espacios liberados
6	Cuarto vehículo de un residente	Rechazado con aviso
7	No socio llega al parqueo	Cae en el área general, nunca en A-C
8	Socio llega al parqueo	Cae en A1
9	Misma placa dos veces	Aviso "ya está estacionada"
10	Salida de placa inexistente	Evento "no estaba estacionado", sin errores
11	Cerrar y reabrir	Residentes y vehículos persisten; estacionados no
12	Ejecución del `.jar` desde carpeta limpia	Abre y carga los datos de ejemplo

12. Ejecución

1. Colocar `parqueo.jar`, `residentes.txt` y `vehiculos.txt` en una misma carpeta.
2. Abrir una terminal en esa carpeta.
3. Ejecutar `java -jar parqueo.jar` (requiere JDK 17 o superior).